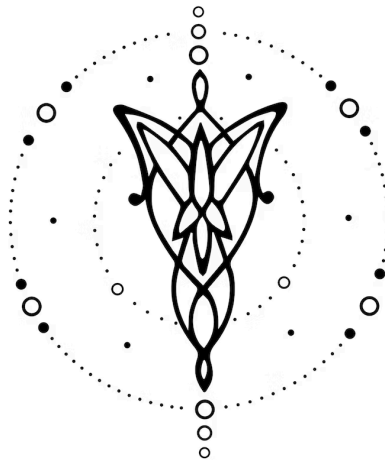




Proyecto Especial

Colorito

1. Introducción	2
2. Modelo computacional	2
2.1 Dominio	2
2.2 Lenguaje	2
3. Implementación	3
3.1 Frontend	3
3.2 Backend	3
3.3 Dificultades encontradas	4
4. Futuras extensiones	4
5. Conclusión	4
6. Referencias	5

1. Introducción

El objetivo de este trabajo es desarrollar un compilador para un lenguaje llamado Colorito, diseñado para aplicar transformaciones visuales sobre imágenes de forma automatizada. El lenguaje permite al usuario escribir instrucciones como `resize`, `crop`, `blend`, entre otras, para modificar imágenes cargadas desde archivos.

El compilador toma como entrada un programa en Colorito y genera como salida un script en Python que aplica las transformaciones indicadas usando una librería auxiliar llamada Pillow. El proceso incluye análisis léxico, sintáctico, semántico y generación de código.

Este proyecto permite automatizar tareas que en programas de edición visual suelen ser repetitivas, haciendo más eficiente su ejecución y facilitando su reutilización.

2. Modelo computacional

2.1 Dominio

Desarrollar un lenguaje que permita realizar alteraciones visuales y generar derivados a partir de una o varias imágenes de entrada. El lenguaje debe permitir modificar aspectos de las imágenes como sus colores, tamaño, orientación, brillo y agregar extras como filtros, pixelados o unir dos imágenes.

Para lograr esto, contaremos con instrucciones dentro de nuestro lenguaje de programación que nos permitirán aplicar estas transformaciones dada una o varias imágenes y parámetros de configuración para la transformación a aplicar. A partir de esto tendremos no solo un lenguaje de programación capaz de aplicar ciertas transformaciones, si no que las transformaciones variarán entre sí dependiendo de los parámetros que se le ingresen.

La implementación satisfactoria de este lenguaje permitiría reducir el tiempo que se pasa en los programas de diseño en tareas básicas, así como también dar la oportunidad de automatizar procesos que normalmente demoraría mucho tiempo a los usuarios que los realizan de forma repetitiva, sería cuestión de realizar un código preciso y ejecutarlo tantas veces como sea necesario.

2.2 Lenguaje

Colorito es un lenguaje de dominio específico (DSL) diseñado para describir transformaciones sobre imágenes de manera simple y declarativa. Su sintaxis está inspirada en lenguajes de alto nivel, con instrucciones compuestas por una palabra clave (como `open`, `resize`, `save`) seguida de los parámetros correspondientes.

En cuanto a las variables el lenguaje soporta distintos tipos (`image`, `string`, `integer`, `percentage`, `dimensión`, `color`), no es necesario especificar el tipo de las variables, se infiere de lo asignado y no se permite la reasignación de las mismas.

Además todas las funciones devuelven imágenes nuevas modificadas, es decir no se modifican las imágenes pasadas por parámetros, permitiendo así su reutilización. Otro de los puntos fuertes de las funciones es que, con el uso de paréntesis, se permite su anidamiento,

haciendo posible encadenar transformaciones, sin la necesidad de guardar los resultados en nuevas variables.

Finalmente se debe tener en cuenta que todas las instrucciones deben terminar en punto y coma (;).

Este diseño del lenguaje prioriza la legibilidad y la facilidad de uso para usuarios que quieran automatizar tareas visuales básicas sin necesidad de escribir código de bajo nivel.

3. Implementación

3.1 Frontend

En el desarrollo del frontend se llevaron a cabo dos etapas principales: primero, la definición del alfabeto del lenguaje Colorito, y luego, el diseño de su gramática. Al tratarse de un lenguaje propio, fue necesario definir desde cero todos los elementos léxicos que serían reconocidos, incluyendo instrucciones específicas del dominio como open, resize, crop, blend, save, entre otras.

Para el análisis léxico se utilizó Flex, donde se implementaron acciones para cada tipo de token esperado. En esta instancia no se presentaron grandes dificultades, ya que el lenguaje fue diseñado con una sintaxis clara y acotada. Se agregaron también logs para facilitar el seguimiento de los tokens ignorados, como espacios o comentarios.

En cuanto al análisis sintáctico, realizado con Bison, se pensaron bien todas las estructuras de manera de permitir lo máximo posible su reutilización, así a su vez se pudieran reutilizar las funciones que las usan. También se tomó la decisión de limitar la complejidad del lenguaje, priorizando expresiones que sean relevantes y útiles para la edición visual.

A partir de la gramática establecida se elaboraron casos de prueba para verificar el correcto reconocimiento de programas válidos y también casos de rechazo, destinados a asegurar que el compilador detecte errores cuando el código fuente no respeta la sintaxis definida.

3.2 Backend

El backend del compilador de Colorito se estructuró en dos fases principales: el análisis semántico y la generación de código. A diferencia del frontend, que se basaba en estructuras fijas y una gramática estática, el backend implicó lógica más compleja y validaciones dinámicas.

En la etapa de análisis semántico, se implementó un sistema de tipos que verifica que cada operación reciba los parámetros adecuados. Por ejemplo, la instrucción resize debe recibir una imagen y una dimensión, mientras que opacity espera un porcentaje. Para esto, se utilizó una tabla de símbolos que almacena el nombre y tipo de cada variable declarada. Esta tabla se recorre en cada instrucción para garantizar que las variables hayan sido definidas y que los tipos coincidan.

Durante esta etapa surgieron varios desafíos, especialmente con el manejo de identificadores que referencian expresiones anidadas y con la validación de combinaciones de parámetros. También fue necesario controlar que no se redeclaren variables con el mismo nombre y que se respeten los tipos en asignaciones.

Una vez superado el análisis semántico, se pasa a la generación de código. Esta fase transforma el árbol de sintaxis abstracta en un archivo .py que puede ejecutarse directamente con Python. El código generado invoca funciones de una librería propia (image_ops.py) que implementa las operaciones gráficas reales haciendo uso de la librería Pillow. Se cuidó especialmente la indentación, el orden de las instrucciones y la correcta anidación de funciones para que el resultado sea sintácticamente correcto y legible.

3.3 Dificultades encontradas

Se presentaron diversas dificultades a lo largo de las distintas etapas del trabajo. Inicialmente, surgieron inconvenientes en la reserva y liberación de memoria, incluyendo problemas de memory leaks, cuyo origen resultó difícil de identificar.

Más adelante, al integrar la aceptación de variables en el lenguaje, fue necesario repensar ciertos aspectos de la estructura del árbol, lo cual resultó complejo en un principio.

Por otro lado, durante la etapa de backend, aparecieron algunos bugs, aunque fueron más fáciles de detectar y no representaron mayores complicaciones para su resolución.

4. Futuras extensiones

Una primera extensión posible sería ampliar la cantidad de operaciones visuales disponibles en el lenguaje. Si bien actualmente se cubren muchas transformaciones comunes como resize, crop, blend o greyscale, podrían incorporarse otras como aplicar bordes, detección de rostros o filtros más avanzados, lo cual implicaría también actualizar la librería de soporte en Python.

Otra mejora interesante sería la posibilidad de trabajar con bloques condicionales o estructuras de repetición, permitiendo definir transformaciones dinámicas sobre múltiples imágenes o condiciones de ejecución. Esto implicaría extender tanto la gramática como el modelo de ejecución del lenguaje.

También consideramos útil permitir procesar carpetas enteras de imágenes, para aplicar scripts en batch sin necesidad de declarar cada imagen una por una. Esto facilitaría mucho su uso en tareas automatizadas.

Por otro lado, una extensión más ambiciosa sería construir una interfaz gráfica que permita escribir código en Colorito, visualizar los pasos intermedios y ver el resultado final en tiempo real, lo cual podría ser especialmente útil para usuarios no familiarizados con programación.

Finalmente, otra funcionalidad posible sería permitir generar directamente archivos .png o .jpg optimizados, integrando más control sobre la exportación y compresión de las imágenes procesadas.

5. Conclusión

Habiendo finalizado el desarrollo de Colorito, estamos muy conformes con el resultado alcanzado. Si bien desde los primeros años de la carrera venimos trabajando con distintos lenguajes de programación, diseñar uno propio desde cero y construir su compilador completo fue un gran desafío y, al mismo tiempo, un logro importante.

El uso de herramientas como Flex y Bison, junto con la implementación del análisis semántico y la generación de código Python, nos permitió entender en profundidad cómo se construye un compilador y cómo interactúan sus distintas etapas. Además, durante el proceso fortalecimos nuestros conocimientos de C, estructura de datos y diseño modular.

Nos vamos de este proyecto con una comprensión mucho más sólida sobre el funcionamiento interno de los compiladores, y con herramientas que sin duda nos van a ser útiles en futuros trabajos tanto académicos como profesionales. También fue muy gratificante ver cómo un lenguaje propio puede traducirse en algo visualmente tangible y funcional, como lo son las transformaciones sobre imágenes.

6. Referencias

- [NumPy](#): Librería fundamental para la computación científica en Python. Utilizada para el manejo de arreglos y operaciones matemáticas.
- [Pillow \(PIL Fork\)](#): Biblioteca de procesamiento de imágenes en Python.
- [logging](#): Módulo estándar de Python para el registro de mensajes de log. Utilizado para depurar y reportar eventos durante la ejecución del programa.